Case Study - 03:

Withdrawal limit invariant - Withdrawal should not exceed balance and overdraft limit; otherwise invalid negative balances may occur.
Frozen account invariant - Frozen accounts cannot perform withdrawals or transfers; otherwise unauthorized transactions may occur.
Positive amount invariant - All transaction amounts must be greater than zero; otherwise account records become incorrect.
Transfer invariant - Transfer must debit and credit both accounts successfully; otherwise money may be lost.
Balance consistency invariant - Account balance must match ledger history; otherwise records become inaccurate.
Unique account number invariant - Every account must have a unique number; otherwise accounts may be confused.

Q2.

```
//Object
class AccountNumber {
    private final String value;

    public AccountNumber(String value) {
        this.value = value;
    }
}

// Value Object
class Money {
    private final double amount;

    public Money(double amount) {
        this.amount = amount;
    }
}

// Value Object
class OverdraftLimit {
    private final Money limit;
```

```
    public OverdraftLimit(Money limit) {
        this.limit = limit;
    }
}

// Entity
class LedgerEntry {
    private String entryId;
    private Money amount;

    public LedgerEntry(String entryId, Money amount) {
        this.entryId = entryId;
        this.amount = amount;
    }
}

// Entity
class BankAccount {
    private AccountNumber accountNumber;
    private Money balance;
    private OverdraftLimit overdraftLimit;

    public BankAccount(AccountNumber accountNumber,
                       Money balance,
                       OverdraftLimit overdraftLimit) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.overdraftLimit = overdraftLimit;
    }
}

// Entity
class FundTransfer {
    private String transferId;
    private BankAccount sourceAccount;
    private BankAccount destinationAccount;
    private Money amount;

    public FundTransfer(String transferId,
```

```
                        BankAccount sourceAccount,
                        BankAccount destinationAccount,
                        Money amount) {
        this.transferId = transferId;
        this.sourceAccount = sourceAccount;
        this.destinationAccount = destinationAccount;
        this.amount = amount;
    }
}

public class Main {
    public static void main(String[] args) {

        // Value Objects
        AccountNumber accNo1 = new AccountNumber("ACC101");
        AccountNumber accNo2 = new AccountNumber("ACC102");

        Money balance1 = new Money(10000);
        Money balance2 = new Money(5000);

        OverdraftLimit limit1 = new OverdraftLimit(new Money(2000));
        OverdraftLimit limit2 = new OverdraftLimit(new Money(1000));

        // Entities
        BankAccount account1 = new BankAccount(accNo1, balance1, limit1);
        BankAccount account2 = new BankAccount(accNo2, balance2, limit2);

        LedgerEntry entry =
                new LedgerEntry("TXN001", new Money(1500));

        FundTransfer transfer =
                new FundTransfer(
                        "FT001",
                        account1,
                        account2,
                        new Money(1500));

        System.out.println("Objects created successfully.");
    }
}
```

Case Study 05:

Valid Email Invariant - Email must be in a valid format; otherwise verification emails cannot be delivered.
Verification Date Invariant - A verified user must have a verification date; otherwise records become inconsistent.
Token Expiry Invariant - Expired tokens cannot be used; otherwise security issues may occur.
Phone Number Format Invariant - Phone numbers must follow a standard format; otherwise duplicate records may appear.
Address Completeness Invariant - Postal addresses must be complete; otherwise communication problems may occur.
Suspended Account Invariant - Suspended users cannot log in; otherwise unauthorized access may occur.
Unique Email Invariant - Each email address must be unique; otherwise duplicate user accounts may be created.

Q2.

```
// Value Object
class UserIdentifier {
    private final String id;

    public UserIdentifier(String id) {
        this.id = id;
    }
}

// Value Object
class EmailAddress {
    private final String email;

    public EmailAddress(String email) {
        this.email = email;
    }
}

// Value Object
class PhoneNumber {
    private final String number;
```

```
    public PhoneNumber(String number) {
        this.number = number;
    }
}

// Value Object
class PostalAddress {
    private String street;
    private String city;
    private String postalCode;

    public PostalAddress(String street, String city, String postalCode) {
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }
}

// Entity
class UserAccount {
    private UserIdentifier userId;
    private EmailAddress email;
    private PhoneNumber phone;
    private PostalAddress address;

    public UserAccount(UserIdentifier userId,
                       EmailAddress email,
                       PhoneNumber phone,
                       PostalAddress address) {
        this.userId = userId;
        this.email = email;
        this.phone = phone;
        this.address = address;
    }
}

// Entity
class VerificationToken {
    private String tokenId;
    private String tokenValue;
```

```
    public VerificationToken(String tokenId, String tokenValue) {
        this.tokenId = tokenId;
        this.tokenValue = tokenValue;
    }
}

public class Email {
    public static void main(String[] args) {

        UserIdentifier userId =
                new UserIdentifier("U101");

        EmailAddress email =
                new EmailAddress("user@gmail.com");

        PhoneNumber phone =
                new PhoneNumber("9876543210");

        PostalAddress address =
                new PostalAddress(
                        "Main Road",
                        "Hyderabad",
                        "500001");

        UserAccount user =
                new UserAccount(
                        userId,
                        email,
                        phone,
                        address);

        VerificationToken token =
                new VerificationToken(
                        "T001",
                        "ABC123XYZ");

        System.out.println("User account created successfully.");
    }
}
```